# SIMATS ENGINEERING
# ASSESSMENT TOOL 1
# ARTIFICIAL INTELLIGENCE (CSA1708)

**Name:** G. Ramani **Reg. No:** 192312279

## 1. Water Jug Problem

### Problem Understanding

Two unmarked jugs have capacities 4 gallons and 3 gallons. Allowed operations are: fill a jug completely from the pump, empty a jug onto the ground, and pour water from one jug to the other until either the source is empty or the destination is full. The objective is to obtain exactly 2 gallons in the 4-gallon jug.

### State-Space Formulation

| Element | Definition |
|---|---|
| State | (x,y), where x = water in 4-gallon jug ($0 \le x \le 4$) and y = water in 3-gallon jug ($0 \le y \le 3$). |
| Initial state | (0,0) |
| Goal test | x = 2; a convenient goal state is (2,0). |
| Operators | Fill 4J, Fill 3J, Empty 4J, Empty 3J, Pour 4J→3J, Pour 3J→4J. |
| Path cost | Each operation may be treated as one step. |

### Solution Process

| Step | Operation | State (4J,3J) |
|---|---|---|
| 0 | Start | (0,0) |
| 1 | Fill 3-gallon jug | (0,3) |
| 2 | Pour 3J → 4J | (3,0) |
| 3 | Fill 3-gallon jug again | (3,3) |
| 4 | Pour 3J → 4J until 4J is full | (4,2) |
| 5 | Empty 4-gallon jug | (0,2) |
| 6 | Pour 3J → 4J | (2,0) |

State sequence: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0).

### Interpretation of Result

The final state is (2,0), so the 4-gallon jug contains exactly 2 gallons. The goal is achieved in 6 valid operations without using any measuring device.

## 2. Mars Rover – Intelligent Agent Analysis

### Problem Understanding

A Mars Rover must autonomously perceive an unknown environment, navigate safely, select scientific targets, collect/analyse samples, manage limited resources and communicate results to Earth. A PEAS-style analysis and task-environment characterization are appropriate.

### i. Percepts

- Camera images of terrain, rocks and navigation hazards.
- Range/proximity and terrain information used for obstacle detection and localization.
- Temperature, atmospheric pressure, radiation, dust/weather and other environmental measurements.
- Rock/soil mineralogical and chemical-composition measurements from scientific instruments.
- Position, orientation, wheel odometry/slip, arm/tool status, battery level, power generation, communication and overall system-health data.

### ii. Operating Environment

| Dimension | Characterization | Reason |
|---|---|---|
| Observability | Partially observable | Sensors provide only limited/local and sometimes noisy information; the complete Martian state is not directly known. |
| Determinism | Stochastic / uncertain | Wheel slip, terrain interaction, dust and sensor uncertainty can make action outcomes unpredictable. |
| Episodic vs sequential | Sequential | Each navigation or sampling decision affects later states, energy and future choices. |
| Static vs dynamic | Dynamic / semi-dynamic | Environmental conditions and resource levels can change while the rover plans and acts. |
| Discrete vs continuous | Mostly continuous | Position, time, energy, temperature and motion vary continuously, although commands may be discrete. |
| Agents | Primarily single-agent | The rover acts autonomously in its local environment, coordinated with mission control. |
| Known vs unknown | Partially unknown | Some terrain properties and scientific characteristics must be discovered during exploration. |

### iii. Actions

- Drive forward/backward, turn, stop and navigate to waypoints.
- Avoid hazards, re-plan paths and select safe terrain.
- Move robotic arm/mast and position scientific instruments.
- Drill, abrade, scoop or collect rock/soil samples where supported by the rover's tools.
- Capture images, perform scientific measurements and analyse samples.
- Store/compress/transmit data and receive high-level commands from Earth.
- Manage battery/power, thermal state, communication and safe-mode operations.

**iv. Performance Evaluation**

A suitable performance measure includes: scientific value and quality of data collected; number of mission objectives completed; navigation safety and hazard avoidance; accuracy/success of sample analysis; energy and time efficiency; rover health and reliability; communication success; useful distance/area explored; and survival within operational constraints.

**v. Suitable Agent Architecture**

A hybrid model-based, goal/utility-based learning architecture is most suitable. A model-based component maintains an internal estimate of the partially observable world; goal-based planning chooses actions that accomplish mission objectives; utility-based reasoning balances scientific value, risk, time and energy; learning/adaptation improves decisions from experience. A reactive layer is also valuable for immediate safety responses such as stopping before a hazard.

**Conclusion**

The rover is best treated as an autonomous rational agent that combines sensing, internal state estimation, planning, utility/risk evaluation, learning and reactive safety control.

## 3. 8-Queens Problem – Search Space and Search Strategy

**Problem Understanding**

The task is to place eight queens on an 8×8 chessboard so that no two queens attack each other. Therefore, no two queens may share a row, column or diagonal.

**Problem Formulation**

| Component | Formulation |
|---|---|
| State representation | A partial placement [r1,r2,...,rk], where ri is the row of the queen in column i and $k \leq 8$. |
| Initial state | Empty board: [] (k=0). |
| Actions / successor | Place one queen in the next column in any row not attacked by already placed queens. |
| Transition model | Append the selected safe row to the partial placement. |
| Goal test | k=8 and all queens are pairwise non-attacking. |
| Path cost | One unit per queen placement; a complete placement has depth 8. |

**Search Space**

If each column could independently choose any of 8 rows, there are 8^8 possible column-wise assignments before constraints are applied. By placing one queen per column and rejecting row/diagonal conflicts immediately, the search space is greatly reduced.

**Method Selection: DFS with Backtracking**

Depth-First Search with backtracking is appropriate because it explores one placement sequence deeply, detects constraint violations early, and returns to the previous decision when a dead end is reached. This avoids storing a very large frontier.

**Safety Test**

For a new queen at (r,c) and an existing queen at (ri,i), it is safe only if $r \neq ri$ and $|r-ri| \neq |c-i|$ for every existing queen. One queen per column automatically eliminates column conflicts.

**Pseudocode**

```
SOLVE_QUEENS(column):
   if column > 8:
      return SUCCESS
   for row = 1 to 8:
      if SAFE(row, column):
         PLACE queen at (row, column)
         if SOLVE_QUEENS(column + 1) = SUCCESS:
            return SUCCESS
         REMOVE queen from (row, column)   // backtrack
   return FAILURE
```

**Example Valid Goal State**

One valid arrangement, listing the row selected for columns 1 to 8, is [1,5,8,6,3,7,2,4]. Hence the queens are at (1,1), (5,2), (8,3), (6,4), (3,5), (7,6), (2,7), (4,8). Every row and column is unique, and no pair satisfies the diagonal-conflict condition.

**Interpretation**

The formulation converts the puzzle into a state-space search problem. DFS with backtracking systematically explores legal partial states and terminates when all 8 queens are placed without conflicts.

## 4. OLA Cab Booking – Problem-Solving Agent

**Problem Understanding**

The customer provides a pickup location, destination and preferences, and the application must choose among available cab categories/routes to achieve the travel goal. This is naturally modeled as a goal-based problem-solving task; utility criteria can refine the choice among alternatives.

**Type of Agent and Justification**

A goal-based problem-solving agent is the primary answer because it identifies a goal state (reach the destination), formulates the problem, searches/evaluates possible actions and executes a solution. When

several alternatives all reach the goal, a utility-based selection can rank them using fare, ETA, travel time, comfort and user preference.

**Problem Formulation**

| Component | OLA Interpretation |
|---|---|
| Initial state | Customer at pickup/source location S. |
| Goal state | Customer reaches destination G. |
| State space | Road locations/intersections together with relevant cab availability/status. |
| Actions | Choose cab category, book/assign cab, move along a road segment, re-route if required. |
| Transition model | An action changes cab/customer location or booking status. |
| Goal test | Current location = destination and trip is completed. |
| Path/utility measure | Distance, fare, ETA/travel time, comfort, availability and possibly traffic. |

**Pseudocode**

```
OLA_PROBLEM_SOLVING_AGENT(source, destination, preference):
   goal ← destination
   cabs ← GET_AVAILABLE_CABS(source)
   if cabs = empty:
      return FAILURE ("No cab available")
   for each cab in cabs:
      estimate pickup_ETA, fare, travel_time and comfort
      compute suitability according to preference
   selected_cab ← BEST_SUITABLE_CAB(cabs)
   route ← SEARCH_BEST_ROUTE(source, destination)
   if selected_cab = NONE or route = NONE:
      return FAILURE
   BOOK(selected_cab)
   ASSIGN_DRIVER(selected_cab)
   EXECUTE(route)
   if current_location = destination:
      return SUCCESS
   else:
      REPLAN and continue
```

**Interpretation**

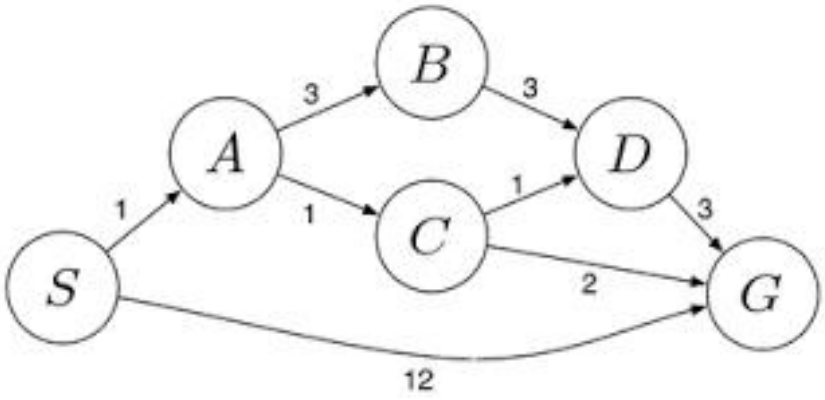
The goal-based agent ensures that actions lead from source to destination, while utility-based ranking helps select the most suitable cab/route when multiple valid choices exist.

**5. Uniform Cost Search (UCS) – Least-Cost Warehouse Route**

**Problem Understanding**

The weighted directed graph represents travel costs between warehouses. The task is to find the path from S to G with minimum cumulative cost. Uniform Cost Search is appropriate because it always expands the frontier node with the smallest path cost g(n) and, with non-negative edge costs, returns an optimal solution.

**Graph / Edge Costs**




| Directed Edge | Cost |
|---|---|
| S → A | 1 |
| S → G | 12 |
| A → B | 3 |
| A → C | 1 |
| B → D | 3 |
| C → D | 1 |
| C → G | 2 |
| D → G | 3 |

**UCS Algorithm**

```
UCS(start, goal):
    frontier ← priority queue containing (start, cost 0)
    best_cost[start] ← 0
    while frontier is not empty:
        node ← remove entry with minimum cumulative cost
        if node = goal:
            return path and cumulative cost
        for each successor of node:
            new_cost ← cost(node) + edge_cost(node, successor)
            if successor is new OR new_cost < best_cost[successor]:
                update best_cost and parent
```

insert/update successor in frontier
return FAILURE

**Step-by-Step Frontier Expansion**

| Step | Node Expanded | g(n) | Frontier after expansion (lowest first) |
|---|---|---|---|
| 0 | S | 0 | A:1, G:12 |
| 1 | A | 1 | C:2, B:4, G:12 |
| 2 | C | 2 | D:3, G:4, B:4 (G updated from 12 to 4) |
| 3 | D | 3 | G:4, B:4 (path to G via D costs 6, so keep G:4) |
| 4 | B or G (tie at 4) | 4 | If B is expanded first, D via B costs 7 and gives no improvement. Then G:4 is selected. |
| 5 | G | 4 | Goal removed from priority queue; terminate. |

**Cost Calculation**

Optimal path: S → A → C → G
Cost = c(S,A) + c(A,C) + c(C,G) = 1 + 1 + 2 = 4 units.

**Verification with Other Complete Routes**

| Path | Total Cost |
|---|---|
| S → G | 12 |
| S → A → C → G | 1+1+2 = 4 |
| S → A → C → D → G | 1+1+1+3 = 6 |
| S → A → B → D → G | 1+3+3+3 = 10 |

**Final Result and Engineering Interpretation**

The least-cost route is S → A → C → G with total cost 4 units. For the logistics company, selecting this route minimizes the modeled combined fuel/time cost among the available routes. UCS is complete and optimal here because all route costs are non-negative.